
Deep Learning Foundations for NLP

1.1 Chapter Overview and Learning Outcomes

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter ??, we now develop the neural computational foundations that allow language models to scale. In contemporary NLP, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over a continuous vocabulary space.

This chapter develops the atomic units of deep learning from the ground up. Beginning with the biological abstraction of the perceptron, we formalize the affine projection and activation mechanism of artificial neurons. We prove rigorously why multi-layer linear networks collapse mathematically into a single linear transformation without non-linear activations, state the Universal Approximation Theorem, and compare the major activation functions used across NLP architectures. We examine loss functions for regression and classification (MSE, BCE, CCE), unpack iterative optimization via Gradient Descent and its adaptive variants, and conduct an exhaustive adjoint trace of backpropagation on a directed acyclic computational graph (DAG), matching the exact formulation taught at Politecnico di Torino.

Upon completing this chapter, you will be able to:

- Formulate the artificial perceptron as an affine transformation followed by a scalar activation function in both scalar and vectorized matrix forms.
- Count neural network parameters rigorously, including distinct weight matrices and bias vectors across stacked layers.
- Prove analytically why any cascade of linear layers without non-linear activations collapses into a single equivalent linear layer ($z = \mathbf{W}^T \mathbf{x}$).
- Analyze the properties, operating ranges, and gradient pathologies of key activation functions (σ , $\tanh$, ReLU, Leaky ReLU, GELU, softmax, and linear identity).
- State and interpret the Universal Approximation Theorem [? ?] and justify the computational efficiency of deep, narrow networks over shallow, wide networks.
- Configure appropriate output activation heads and loss functions for binary classification, multi-class classification, and regression.
- Trace forward intermediate values and backward adjoint gradients ($\partial \mathcal{L} / \partial \cdot$) across a computational DAG using the multivariate chain rule [?].
- Explain the limitations of vanilla Gradient Descent in non-convex loss landscapes and the mechanics of Momentum and Adam optimizers.

1.2 From Biological Neurons to the Artificial Perceptron

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrical impulses from neighboring cells through branching *dendrites*, integrates these incoming signals in its central cell body (*soma*), and, when the accumulated electrical potential exceeds a physiological threshold, fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism is formalized as a linear combination of input features weighted by synaptic connection strengths, augmented by an additive bias, and passed through an activation function.

Definition 1.1: The Artificial Perceptron

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f \left(w_0 + \sum_{i=1}^n w_i x_i \right) = f (w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n), \quad (1.1)$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \dots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (1.1) is expressed in compact vector inner-product notation:

$$y = f \left(\sum_{i=0}^n w_i x_i \right) = f (\mathbf{w}^\top \mathbf{x}) \quad \text{with } x_0 = 1, w_0 = \text{bias}. \quad (1.2)$$

Geometric Interpretation in Two Dimensions. When $n = 2$, the input consists of two spatial coordinates $\mathbf{x} = [x_1, x_2]^\top$. The pre-activation affine combination:

$$z = w_1 x_1 + w_2 x_2 + w_0 = 0 \quad (1.3)$$

defines a straight line in the Cartesian plane $\mathbb{R}^2$. This line acts as a linear *decision boundary*. Points on one side of the line produce $z > 0$, while points on the opposite side produce $z < 0$. The normal vector to the hyperplane is $\mathbf{w}_{1:2} = [w_1, w_2]^\top$, which determines its orientation, while the bias w_0 dictates its perpendicular displacement from the origin.

Connection to Classical Linear and Logistic Regression. The artificial perceptron unifies two foundational statistical models:

- If the activation function is the linear identity $f(z) = z$, Equation (1.2) is exactly **Linear Regression**: $y = \mathbf{w}^\top \mathbf{x}$.
- If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (1.2) is exactly **Logistic Regression**: $y = \sigma(\mathbf{w}^\top \mathbf{x})$, predicting the Bernoulli probability of the positive class.

1.3 Activation Functions and the Proof of Linear Stacking Collapse

Activation functions serve three indispensable architectural roles in neural networks:

1. **Enforcing Output Properties:** Bounding outputs to valid probability intervals (e.g., $[0, 1]$ via σ or the probability simplex via softmax).
2. **Introducing Non-Linearities:** Enabling deep networks to learn complex, non-linear manifolds and preventing the mathematical collapse of stacked layers.
3. **Regulating Gradient Flow:** Supporting stable backpropagation without numerical saturation or catastrophic gradient cancellation.

1.3.1 Fully Connected Linear Layers

In a multi-neuron layer, multiple perceptrons operate in parallel on the shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$. Consider a layer with two distinct outputs:

$$y_1 = f(\mathbf{w}^\top \mathbf{x}), \quad y_2 = f(\mathbf{q}^\top \mathbf{x}). \quad (1.4)$$

Grouping the weight vectors as columns of a weight matrix $\mathbf{W} = [\mathbf{w}, \mathbf{q}] \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and defining bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the operation generalizes to:

$$\mathbf{y} = f(\mathbf{W}^\top \mathbf{x} + \mathbf{b}). \quad (1.5)$$

Exam Insights: Parameter Counting Across Dense Layers

A standard oral and written examination question tests exact parameter counting across dense layers. For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$:

- Weight parameters: $d_{\text{in}} \times d_{\text{out}}$.
- Bias parameters: d_{out} .
- Total parameters: $d_{\text{out}}(d_{\text{in}} + 1)$.

For example, a linear layer with 2 inputs and 2 outputs contains $2 \times 2 = 4$ weights plus 2 biases, giving exactly 6 trainable parameters. Neglecting bias terms during parameter inventory counts is a frequent source of student errors.

1.3.2 The Mathematical Proof of Linear Stacking Collapse

A student encountering neural networks for the first time might ask: *Why can we not simply stack dozens of linear layers without activation functions to build a deep model?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 1.1: Linear Stacking Collapse

Let $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \rightarrow \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1, \quad (1.6)$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2. \quad (1.7)$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned}\mathbf{z} &= \mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + (\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2) \\ &= \mathbf{W}'^\top \mathbf{x} + \mathbf{b}',\end{aligned}\tag{1.8}$$

where $\mathbf{W}' = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n \times k}$ and $\mathbf{b}' = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Proof. Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ is simply a single matrix $\mathbf{W}' \in \mathbb{R}^{n \times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}' \in \mathbb{R}^k$. By induction, any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top (\dots (\mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \dots) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}$$

is algebraically equivalent to a single linear layer. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. ■

💡 Intuition & Analogy: Why Non-Linearity Bends Space

Linear transformations can only rotate, reflect, scale, and shear Euclidean space; they preserve parallel lines and planar flat manifolds. Real-world natural language patterns—such as sentiment classification, syntactic hierarchies, and contextual polysemy—are linearly inseparable. Non-linear activation functions act as topological “folds” and “twists” on the input space, allowing deep neural networks to carve complex, highly non-linear decision boundaries that separate entangled classes.

1.3.3 Taxonomy of Neural Activation Functions

Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads:

1. Logistic Sigmoid. Squashes real-valued inputs $\mathbb{R}$ into the bounded probability interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.\tag{1.9}$$

Its derivative expresses neatly in terms of the function itself:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).\tag{1.10}$$

The maximum derivative occurs at $z = 0$, where $\sigma'(0) = 0.25$. For $|z| \gg 0$, $\sigma'(z) \rightarrow 0$, causing severe **gradient saturation** that halts backpropagation in deep feedforward stacks.

2. Hyperbolic Tangent (tanh). Squashes real inputs into the zero-centered symmetric interval $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1.\tag{1.11}$$

Its derivative is $\tanh'(z) = 1 - \tanh^2(z)$, with maximum value $\tanh'(0) = 1.0$. Because it is zero-centered, it is preferred over sigmoid in recurrent hidden state transitions.

3. Rectified Linear Unit (ReLU). Introduced to mitigate vanishing gradients in deep networks:

$$\text{ReLU}(z) = \max(0, z). \quad (1.12)$$

Its derivative is a piecewise step function: $\text{ReLU}'(z) = 1$ if $z > 0$, and $\text{ReLU}'(z) = 0$ if $z < 0$. It is scale-invariant and computationally cheap, but susceptible to the **dying ReLU** pathology: if a large gradient updates weights such that a neuron outputs $z < 0$ across the entire dataset, its gradient is permanently 0, rendering the neuron inactive.

4. Leaky ReLU. Prevents dying neurons by introducing a small non-zero slope $\alpha \approx 0.01$ in the negative regime:

$$\text{LeakyReLU}(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}. \quad (1.13)$$

5. Gaussian Error Linear Unit (GELU). The standard activation function in modern Transformers (BERT, GPT-2, GPT-3, LLaMA) [? ?]:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \mathbb{P}(X \leq z), \quad X \sim \mathcal{N}(0, 1). \quad (1.14)$$

GELU weights inputs by their percentile value under a standard Gaussian cumulative distribution. It is smooth, non-monotonic, and approximated in deep learning libraries as:

$$\text{GELU}(z) \approx 0.5z \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right) \right). \quad (1.15)$$

6. Softmax. Maps an unnormalized C -dimensional logit vector $\mathbf{z} \in \mathbb{R}^C$ to the probability simplex:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad \text{guaranteeing } \text{softmax}(\mathbf{z})_i \in [0, 1] \text{ and } \sum_{i=1}^C \text{softmax}(\mathbf{z})_i = 1. \quad (1.16)$$

7. Linear Identity Activation. Defined as $f(z) = z$. Typically used exclusively in continuous regression output heads where the predicted quantity is an unbounded real scalar.

1.4 Universal Approximation Theorem

The theoretical foundation validating neural networks as general-purpose function approximators is the Universal Approximation Theorem.

Theorem 1.2: Universal Approximation Theorem

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid). Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n -dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous functions on I_n . Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer

weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^M \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) \quad (1.17)$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} |F(\mathbf{x}) - g(\mathbf{x})| < \epsilon. \quad (1.18)$$

Originally proved by George Cybenko [?] for sigmoidal activations and generalized by Kurt Hornik et al. [?] to arbitrary non-polynomial continuous activations, this theorem guarantees that a feedforward network with a single hidden layer has the capacity to approximate any continuous function.

🔗 External Deep Dive: Why Deep Networks if Shallow Networks are Universal Approximators?

If a single hidden layer can approximate any function, why does modern NLP exclusively use deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

1. **Non-Constructive Existence:** The theorem is purely an existence proof. It guarantees that parameters exist, but provides no algorithm to discover them and establishes no finite bound on the width M .
2. **Exponential Parameter Inefficiency:** Approximating highly compositional functions with a single hidden layer requires width M that scales exponentially with input dimensionality ($M \sim \mathcal{O}(2^n)$).
3. **Hierarchical Compositionality:** Deep networks factorize functions into hierarchical abstractions: early layers learn low-level primitives (subwords, syntax), intermediate layers learn grammatical structure, and deep layers capture long-range semantic discourse. Deep networks achieve equal approximation power with exponentially fewer parameters than shallow networks.

1.5 Output Heads, Loss Functions, and Optimization

1.5.1 Architectural Output Heads

Depending on the learning task, the final hidden representation $\mathbf{h} \in \mathbb{R}^d$ is projected via an affine head $\mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} + \mathbf{b}_{\text{out}}$ into task-specific predictions:

- **Binary Classification:** The model outputs a single scalar logit $z \in \mathbb{R}$. Applying the sigmoid activation yields predicted probability $\hat{y} = \sigma(z) \in [0, 1]$. By convention, $\hat{y} \geq 0.5$ predicts the positive class ($y = 1$), and $\hat{y} < 0.5$ predicts the negative class ($y = 0$).
- **Multi-Class Classification:** For C mutually exclusive categories (such as vocabulary tokens), the model emits a vector of C unnormalized logits $\mathbf{z} = [z_1, \dots, z_C]^\top \in \mathbb{R}^C$. The softmax operator converts logits into a normalized categorical distribution: $\hat{y}_i = \mathbb{P}(y = c_i | \mathbf{x}) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$.

- **Regression:** The model predicts unbounded continuous real numbers. The output head uses the identity activation $f(z) = z$, producing $\hat{y} \in \mathbb{R}$.

1.5.2 Loss Function Formulations

Training a neural network requires defining an objective function $\mathcal{L}(\theta)$ that quantifies the discrepancy between model predictions $\hat{y}$ and ground-truth targets y . Although the loss depends on the training dataset (X, y) , the training data is fixed; thus, $\mathcal{L}$ is framed purely as a function of the parameter vector θ :

1. Mean Squared Error (MSE) for Regression. For continuous target values, the standard objective is the quadratic loss:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \theta^\top \mathbf{x}_i)^2. \quad (1.19)$$

For simple linear regression, the loss surface is strictly convex and quadratic. Setting the gradient $\nabla_{\theta} \mathcal{L}_{\text{MSE}} = 0$ yields the celebrated closed-form *normal equations*:

$$\theta^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (1.20)$$

2. Binary Cross-Entropy (BCE). For binary classification with ground-truth label $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0, 1]$:

$$\mathcal{L}_{\text{BCE}}(\theta) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \quad (1.21)$$

The ground truth y functions as a dynamic *term selector*:

- When $y = 1$, the right term vanishes: $\mathcal{L} = -\log(\hat{y})$. If the model predicts $\hat{y} \approx 1$, the loss is near zero. If $\hat{y} \rightarrow 0$, the loss penalizes the model with $\mathcal{L} \rightarrow \infty$.
- When $y = 0$, the left term vanishes: $\mathcal{L} = -\log(1 - \hat{y})$. If the model predicts $\hat{y} \approx 0$, the loss is near zero. If $\hat{y} \rightarrow 1$, the loss approaches infinity.

3. Categorical Cross-Entropy (CCE). For multi-class classification over C classes, target y is represented as a one-hot vector $\mathbf{y} \in \{0, 1\}^C$ where $y_c = 1$ for the true class and $y_j = 0$ for all $j \neq c$. The Categorical Cross-Entropy loss is:

$$\mathcal{L}_{\text{CE}}(\theta) = - \sum_{i=1}^C y_i \log(\hat{y}_i) = -\log(\hat{y}_{\text{target}}) = -\log \left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^C e^{z_j}} \right). \quad (1.22)$$

Just as in BCE, the one-hot target vector $\mathbf{y}$ acts as a selector, isolating the negative log-probability of the true target token.

1.5.3 Gradient Descent and Optimization Landscapes

Unlike simple linear regression, deep neural networks contain billions of non-linear parameter interactions, rendering analytical closed-form solutions impossible. Parameters must be optimized iteratively via *Gradient Descent*.

Starting from an initial parameter state $\theta^{(0)}$ (typically sampled from a zero-mean Gaussian distribution), the algorithm iteratively updates weights along the direction of steepest descent:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{(t)}), \quad (1.23)$$

where $\alpha > 0$ is the *learning rate*, controlling the spatial step size taken in parameter space.

Pathologies of Vanilla Gradient Descent. In deep, non-convex optimization landscapes, basic gradient descent suffers from several severe limitations:

1. **Sensitivity to Weight Initialization:** In non-convex terrain, different initializations lead to distinct local minima. (In contrast, for convex objectives, any initialization converges to the global minimum).
2. **Local Minima and Saddle Points:** High-dimensional loss surfaces are dominated by saddle points where the gradient vanishes ($\nabla \mathcal{L} = 0$) despite high surrounding curvature.
3. **Oscillations in Ravines:** In regions where curvature is steep along one dimension and shallow along another, gradient descent oscillates erratically across the valley rather than progressing down the floor.

Algorithmic Remedies: Momentum and Adam. To stabilize optimization, modern deep learning utilizes adaptive gradient algorithms:

- **Momentum:** Accumulates an exponential moving average of past gradients $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t)$, carrying the optimization vector through saddle points and damping high-frequency perpendicular oscillations.
- **Adam (Adaptive Moment Estimation):** Maintains exponential moving averages of both past gradients (first raw moment $\mathbf{m}_t$) and past squared gradients (second uncentered moment $\mathbf{v}_t$), incorporating bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (1.24)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (1.25)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \quad (1.26)$$

Adam adaptively scales coordinate step sizes inversely proportional to the root of the accumulated gradient variances.

1.6 Computational Graphs and Exact Adjoint Backpropagation

To compute the gradient vector $\nabla_{\theta} \mathcal{L}$ efficiently across networks containing millions of parameters, modern frameworks (such as PyTorch and TensorFlow) represent the entire mathematical sequence of operations as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 1.2: Computational Graph

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical operation (e.g., $+$, $\times$, $-$, $\exp$).
- Each directed edge $e = (u, v) \in \mathcal{E}$ represents the flow of intermediate data from variable u into operation v .

Elementary DAG Example: $y = wx + q$. Consider evaluating $y = wx + q$:

1. Inputs: variables w , x , and q .
2. First intermediate operation: $a = w \times x$ (multiplication node).
3. Second operation: $y = a + q$ (addition node).

1.6.1 Exact Backpropagation Walkthrough from Slide 24

We now reproduce and derive the exact pedagogical computational graph and adjoint trace from the lecture slides.

Problem Setup. Consider a toy model with two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on a single training example $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction: } \hat{y} = \theta_1 \theta_2 x, \quad (1.27)$$

$$\text{Objective Loss: } \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \quad (1.28)$$

Forward Adjoint Decomposition. The computational graph decomposes Equation (1.28) into four primitive operations and intermediate variables:

$$a = \theta_1 \theta_2 \quad (\text{parameter interaction product}), \quad (1.29)$$

$$b = ax = \theta_1 \theta_2 x \quad (\text{model output prediction } \hat{y}), \quad (1.30)$$

$$c = b - y = \theta_1 \theta_2 x - y \quad (\text{residual prediction error}), \quad (1.31)$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad (\text{scalar squared error loss}). \quad (1.32)$$

Backward Chain Rule Derivation (Adjoint). Backpropagation computes the derivative of the scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by applying the calculus chain rule $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$ in reverse topological order:

$$\text{Step 1: Derivative w.r.t. } c : \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial (c^2)}{\partial c} = 2c. \quad (1.33)$$

$$\text{Step 2: Derivative w.r.t. } b : \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial (b - y)}{\partial b} = 2c \cdot 1 = 2c. \quad (1.34)$$

$$\text{Step 3: Derivative w.r.t. } a : \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial (ax)}{\partial a} = 2c \cdot x = 2cx. \quad (1.35)$$

$$\begin{aligned} \text{Step 4: Derivative w.r.t. } \theta_1 : \frac{\partial \mathcal{L}}{\partial \theta_1} &= \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 \\ &= 2(\theta_1 \theta_2 x - y)x\theta_2. \end{aligned} \quad (1.36)$$

$$\begin{aligned} \text{Step 5: Derivative w.r.t. } \theta_2 : \frac{\partial \mathcal{L}}{\partial \theta_2} &= \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 \\ &= 2(\theta_1 \theta_2 x - y)x\theta_1. \end{aligned} \quad (1.37)$$

Figure 1.1 illustrates this complete forward and backward adjoint execution.

💡 Intuition & Analogy: 3Blue1Brown's Geometric View of Backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus

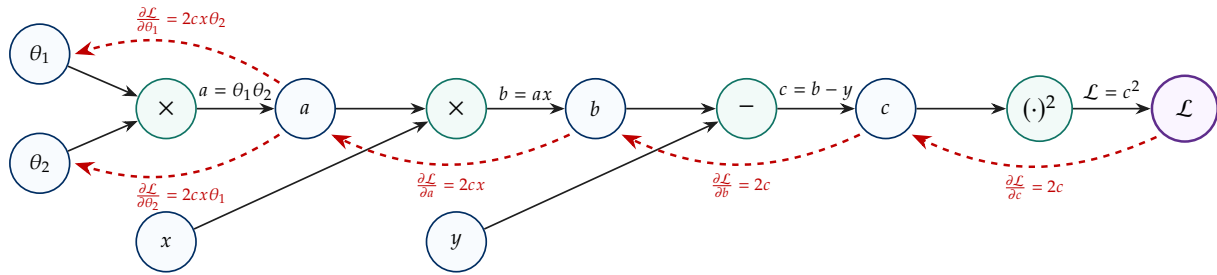


Figure 1.1: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1 \theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1 \theta_2 x - y)^2$ (reproduced from Slide 24). Solid black arrows trace the forward operational flow. Dashed red arrows trace reverse-mode adjoint backpropagation using the chain rule.

algebra, but as a systematic tracking of sensitivity *nudges*. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ . That nudge ripples through the product node to scale a by $\epsilon \theta_2$. The change in a wiggles b by $\epsilon \theta_2 x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c \epsilon \theta_2 x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

External Deep Dive: Building Autograd from Scratch: Andrej Karpathy's Micrograd

Andrej Karpathy's open-source library *Micrograd* implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure 1.1.

Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial \mathcal{L} / \partial \cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to the operand children nodes.
- `._backward`: a lambda closure storing the local calculus chain rule.

For example, multiplication is implemented as:

```
1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4     def _backward():
5         self.grad += other.data * out.grad
6         other.grad += self.data * out.grad
7     out._backward = _backward
8     return out
```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish a reverse dependency ordering, sets `loss.grad = 1.0`, and sequentially executes `node._backward()` across all parent nodes. Modern deep learning frameworks like PyTorch execute this exact process on multi-dimensional GPU tensors.